



BANGALORE SAHODAYA SCHOOLS COMPLEX ASSOCIATION

PRE-BOARD EXAMINATION -1 (2025 – 2026)

GRADE XII

Date: 10.12.2025

Max Marks: 70

Subject: COMPUTER SCIENCE (083)

Time: 3 hours

ANSWER KEY

GENERAL INSTRUCTIONS

- This question paper contains 37 questions.
- All questions are compulsory. However, internal choices have been provided in some questions. Attempt only one of the choices in such questions.
- The paper is divided into 5 Sections- A, B, C, D and E.
- Section A consists of 21 questions (1 to 21). Each question carries 1 Mark.
- Section B consists of 7 questions (22 to 28). Each question carries 2 Marks.
- Section C consists of 3 questions (29 to 31). Each question carries 3 Marks.
- Section D consists of 4 questions (32 to 35). Each question carries 4 Marks.
- Section E consists of 2 questions (36 to 37). Each question carries 5 Marks.
- All programming questions are to be answered using Python Language only.
- In case of MCQ, text of the correct answer should also be written.

Q.NO	SECTION -A (21 x 1=21 Marks)	MARKS
1	False	1
2	c) ('Welcome', 'To', ' pythonWorld')	1
3	b) 515	1
4	b) print(dict1(["Sam", "Ram"]))	1
5.	c) General Error Done	1
6	d) WHERE → GROUP BY → HAVING → SELECT → ORDER BY	1
7	b) 30	1
8	b) 11@6	1
9	c) ALTER TABLE and INSERT	1
10	d) ie	1
11	b) Cpecn26	1
12	a) 8	1
13	b) SFTP	1
14	a) True	1
15	a) gimroPnhy	1
16	c) It determines the optimal path for data packets between different networks.	1
17	b) Fiber Optic Cable	1

18	c) Like “UP%”	1
19	c) Drop table Students	1
Q20 and Q21 are Assertion(A) and Reason(R) based questions. Mark the correct choice as: a) Both A and R are True and R is the correct explanation for A. b) Both A and R are True and R is not the correct explanation for A. c) A is True but R is False. d) A is False but R is True.		
20	a) Both A and R are True and R is the correct explanation.	1
21	b) Both A and R are True and R is not the correct explanation.	1
SECTION-B (7 x 2 = 14)		
22	A) Keyword: Reserved word, cannot be used as a name. Examples: if, while, for, def Identifier: User-defined name for variables, functions, etc. Examples: total, sum1, StudentName, calc_area OR B) Logical operators: and, or, not – used to combine conditions. Relational operators: ==, !=, >, <, >=, <= – used for comparisons. (1 mark for correct difference , 1 mark for correct examples))	2
23	def remove_vowels(text): vowels = "aeiouAEIOU" new_text = " count = 0 for char in text: if char not in vowels: new_text += char else: count += 1 return new_text, count original = "Education" result, num = remove_vowels(original) print(result, num) (1/2 x 4 = 2 Marks for each correct error)	2
24	A) I. s = s.strip().title() II. last3 = str[-3:].upper() OR B) Using Python built-in functions only: I) L = [1, 2, 3, 4] L.reverse() II) s = "INFORMATICS" sub = s[5:10]	1+1

25	cbse@board@#202@-2 (2 marks for correct output)	2		
26	4 (2 marks for correct output)	2		
27	A. i. ALTER TABLE SUPPLIER MODIFY phone_no INT; ii. DELETE FROM STUDENT; (1 marks for correct query) OR B. <table><tr><td>WHERE - Filters rows before grouping. - Cannot use aggregate functions. Example: SELECT Department, Salary FROM EMP WHERE Salary > 50000;</td><td>HAVING - Filters after GROUP BY. - Works with aggregate functions. Example: SELECT Department, AVG(Salary) FROM EMP GROUP BY Department HAVING AVG(Salary) > 60000;</td></tr></table> (1 mark for correct difference , 1 mark for correct example)	WHERE - Filters rows before grouping. - Cannot use aggregate functions. Example: SELECT Department, Salary FROM EMP WHERE Salary > 50000;	HAVING - Filters after GROUP BY. - Works with aggregate functions. Example: SELECT Department, AVG(Salary) FROM EMP GROUP BY Department HAVING AVG(Salary) > 60000;	2
WHERE - Filters rows before grouping. - Cannot use aggregate functions. Example: SELECT Department, Salary FROM EMP WHERE Salary > 50000;	HAVING - Filters after GROUP BY. - Works with aggregate functions. Example: SELECT Department, AVG(Salary) FROM EMP GROUP BY Department HAVING AVG(Salary) > 60000;			
28	A. (i) IMAP – Internet Message Access Protocol (ii) ISP – Internet Service Provider (1 mark each for correct expansion) OR B. Intranet: Private network within an organization, accessible only to authorised members. Internet: Global network of networks, publicly accessible. (1 mark each for correct difference)	1+1		
Section-C (3 x 3 = 9 Marks)				
29	A. def count_lines(): count = 0 with open("Quote.txt", "r") as f: lines = f.readlines() for line in lines: words = line.split() if "programming" in words or "coding" in words: count += 1 print("Number of lines:", count) count_lines() (1/2 mark for correctly opening the file) (1/2 mark for correctly reading from the file)	3		

	(1/2 mark for splitting the text into words) (1/2 mark for if condition checking text) (1 mark for correctly display the desired words) OR B. <pre>def show_digit_lines(): with open("LOG.TXT", "r") as f: data = f.read() lines = data.split("\n") # data.splitlines() also correct for line in lines: if line and line[0].isdigit(): print(line)</pre> show_digit_lines() (1/2 mark for correct function header) (1/2 mark for correctly opening the file) (1/2 mark for correctly reading from the file) (1/2 mark for splitting the text into words) (1 mark for correctly displaying the count)	
30	<pre>def push_Book(BookStack, new_Book): BookStack.append(new_Book)</pre> <pre>def pop_Book(BookStack): if len(BookStack) == 0: print("Underflow - No book to remove") return None else: return BookStack.pop()</pre> BookStack = [] push_Book(BookStack, ['Python Basics', 'Guido', 499]) push_Book(BookStack, ['Data Science', 'Andrew', 650]) print(pop_Book(BookStack)) (1 ½ mark Function definition and push logic using append()) (1 ½ mark Pop function using pop() with underflow condition) OR <pre>L = [("S001","Amit",92,"CS"), ("S002","Priya",87,"Bio"), ("S003","Raj",78,"CS"), ("S004","Sara",95,"Maths")]</pre>	3

	<pre> TopScorers = [] def PushTopScorers(): for s in L: if s[2] > 85: TopScorers.append((s[0], s[1], s[2])) def PopAndDisplay(): if len(TopScorers)==0: print("Stack Underflow") else: for i in range(len(TopScorers)): rec = TopScorers.pop() print("ID:",rec[0]," Name:",rec[1]," Marks:",rec[2]) print("Stack Underflow") PushTopScorers() PopAndDisplay() </pre> (1 ½ mark Function definition and push logic using append()) (1 ½ mark Pop function using pop() with output statement)	
31	a) CODE44041010DATA (3 marks for the correct output) OR b) AI10#ML9#DL8 (3 marks for the correct output)	3
Section-D (4 x 4 = 16 Marks)		
32	<pre> import mysql.connector as mc def Add(): con = mc.connect(host="localhost", user="root", password="Library", database="LIBRARY") cur = con.cursor() bno = int(input("Enter Book No: ")) bname = input("Enter Book Name: ") price = float(input("Enter Price: ")) qty = int(input("Enter Quantity: ")) q = "INSERT INTO BOOK (BNo, BName, Price, Qty) VALUES (%s, %s, %s, %s)" cur.execute(q, (bno, bname, price, qty)) con.commit() con.close() def Display(): con = mc.connect(host="localhost", user="root", passwd="Library", </pre>	4

	<pre> database="LIBRARY") cur = con.cursor() q = "SELECT * FROM BOOK WHERE Price > 180" cur.execute(q) data = cur.fetchall() for row in data: print(row) con.close() </pre> (1/2 mark for correctly importing the connector object) (1/2 mark for correctly creating the connection object) (1/2 mark for correctly creating the cursor object) (1 mark for correct creation of update query) (1 mark for correctly executing the query with commit) (1/2 mark for correctly closing the connection)	
33	I) <pre> import csv def AddItem(): item_id = input("Item ID: ") item_name = input("Item Name: ") qty = int(input("Quantity: ")) cost = float(input("Cost per item: ")) if qty > 0 and cost > 0: with open("Stock.csv", "a", newline="") as f: w = csv.writer(f) w.writerow([item_id, item_name, qty, cost]) else: print("Quantity and Cost must be > 0") </pre> (1/2 mark for correctly taking user input) (1/2 mark for opening the file in append mode) (1/2 mark for correctly creating the writer object) (1/2 mark for correctly using writerow() of writer object) II) def TotalCost(): <pre> total = 0 with open("Stock.csv", "r", newline="") as f: r = csv.reader(f) for row in r: qty = int(row[2]) cost = float(row[3]) total += qty * cost print("Total Stock Cost =", total) </pre> (1/2 mark for opening in the file in right mode) (1/2 mark for correctly creating the reader object) (1/2 mark for correctly checking the condition)	4

(1/2 mark for correctly displaying the total sales)
Note (for both parts (I) and (II)): Ignore import csv as it may be considered the part of the complete program

34

A. Write SQL queries for the following:

I. SELECT Name, Department, Salary FROM EMPLOYEES WHERE Salary BETWEEN 60000 AND 90000 AND Department IN ('HR', 'Finance');

II. SELECT Department, COUNT(*) AS EmpCount FROM EMPLOYEES GROUP BY Department HAVING COUNT(*) > 2;

III. SELECT Name, City, Salary * 1.08 AS UpdatedSalary FROM EMPLOYEES WHERE City LIKE 'M%';

IV. SELECT Emp_ID, Name, Salary FROM EMPLOYEES WHERE Join_Date > '2020-12-31' ORDER BY Salary DESC;

(4 x 1 mark for each correct query)

OR

B. Predict the output of the following SQL queries (Assume suitable data in EMPLOYEES table):

I.

Department	EmpCount
Sales	3
IT	3

II.

Name	Department	Bonus
Rajesh Kumar	Sales	8500
Vikram Singh	Sales	4500
Karan Mehta	Sales	8800

III.

City	MinSal	MaxSal
Mumbai	45000	88000
Delhi	72000	75000
Bangalore	48000	95000

IV.

Name	Designation	Salary
Sneha Reddy	Lead	95000
Karan Mehta	Manager	88000
Rajesh Kumar	Manager	85000
Priya Patel	Manager	80000

(4 x 1 mark for each correct output)

4

35	Write SQL queries for the following: I. UPDATE COURSES SET Credits = 5 WHERE Course_ID = 'C105'; II. SELECT E.Student_Name, C.Course_Name, E.Grade, E.Semester FROM COURSES C JOIN ENROLLMENTS E ON C.Course_ID = E.Course_ID WHERE E.Semester = 'III'; III. DELETE FROM ENROLLMENTS WHERE Grade = 'C'; IV. A. SELECT E.Student_Name, C.Course_Name, C.Department FROM COURSES C JOIN ENROLLMENTS E ON C.Course_ID = E.Course_ID WHERE C.Instructor LIKE '%Smith'; OR B. SELECT Department,COUNT(*) AS TotalCourses, AVG(Credits) AS AvgCredits FROM COURSES GROUP BY Department HAVING AVG(Credits) > 3.0; (4 x 1 mark for each correct query)	4
Section - E (2x5 = 10 marks)		
36	<pre>import pickle I) def ADD_Data(): f = open("Student.Dat", "ab") while True: adm = int(input("Admission Number (0 to stop): ")) if adm == 0: break name = input("Name: ") perc = float(input("Percentage: ")) rec = [adm, name, perc] pickle.dump(rec, f) f.close()</pre> (1/2 mark for correctly defining the function header) (1/2 mark for correctly opening the file in append mode) (1/2 mark for correctly taking user input) (1/2 mark for using dump() method of the pickle module)	5

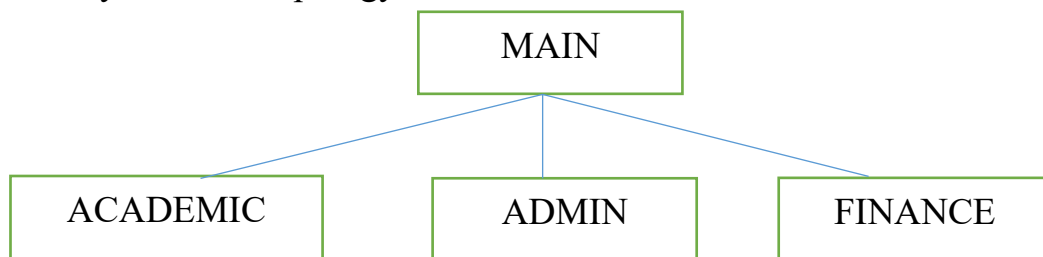
b) def CountRec():
 low = float(input("Enter lower %: "))
 high = float(input("Enter upper %: "))
 count = 0
 try:
 f = open("Student.Dat", "rb")
 while True:
 rec = pickle.load(f)
 if low <= rec[2] <= high:
 print("AdmNo:", rec[0], "Name:", rec[1], "Percentage:",
rec[2])
 count += 1
 except EOFError:
 f.close()
 print("Total records in range:", count)

(1/2 mark for correctly defining the function header)
(1/2 mark for correctly opening the file)
(1 mark for using load() with while loop and try-except block)
(1 mark for checking the condition and displaying the value)

Note (for both parts (a) and (b)): (a) Ignore import pickle as it may be considered the part of the complete program.

37

- (a) Main Building
 (b) Switch
 (c) Cable layout & medium
 • Layout: Star Topology



cable: Fiber Optic cable

- (d) Given distances (≤ 100 m between buildings) and especially with Fiber Optic, **no repeater is required** because the distances are within the permissible range.
 (e) (a) Video Conferencing
 OR
 (b) **WAN** (Wide Area Network).

5